Systems Programming and Computer Architecture

Laurin Seeholzer

October 7, 2026

Contents

1 C Programming **3**
1.1 History and Toolchain . . . 3
1.2 Control flow . . . 3
1.3 Basic types in C . . . 4
1.4 Operators . . . 5
1.5 Arrays . . . 5
1.6 Preprocessing . . . 6
1.7 Modularity . . . 7
1.8 Assertions . . . 8
1.9 Style . . . 8

2 Integers **9**
2.1 Operations . . . 9

3 Pointers **11**
3.1 Memory Layout and Virtual Address Space . . . 11
3.2 Pointer Fundamentals . . . 11
3.3 Pointer Operations . . . 12
3.4 Function Pointers . . . 12

4 Dynamic Memory Allocation **13**
4.1 Data Structures . . . 13

1 C Programming

1.1 History and Toolchain

C is very fast, offers preprocessing and is close to metal. What you won't find in C are objects, classes, traits, features, methods, interfaces, fancy types, exception handling.

Important 1.1. Most importantly C does not offer memory managment, it uses pointers that offer direct access to memory adresses.

We'll use C99, the most commonly used variant, in this course.

Code Example 1.2

```
#include <studio.h>

int main(int argc, char *argv[])
{
    printf("hello, world\n");
    return 0;
}
```

1.2 Control flow

Code Example 1.3

```
if (Boolean expression) Statement_when_true
    else Statement_when_false

switch (Integer expression) {
    case Constant_1: Statement; break;
    ...
    default: Statement; break;
}

for (Initial; Test; Increment) Statement

while (Boolean expression) Statement

do Statement while (Boolean expression)

goto Label

printf()

return (Expression)
```

Important 1.4. Don't use goto: It's almost never a good idea, usually argued on performance.

Remark 1.5. goto can be used to break out of multiple nested loops or to cleanup nested code.

1.3 Basic types in C

Code Example 1.6

```
{static, const} type vairable = value;
```

Remark 1.7. The prefix **const** before a variable ensures it can not be changed lateron.

Definition 1.8. Scope: The scope of a variable in C is the Block it is definied in (lexically typed).

Remark 1.9. The prefix **static** before a variable definition ensures the value persists between function calls. Note that static limits the scope of a variable to the file/compilation unit.

Definition 1.10. Integers & Floats: Size of these differ from system to system

- **char** is always 1 Byte
- **int & float** are always 4 Bytes
- **long long & double** are always 8 Bytes

Definition 1.11. Booleans: Boolean values in C are integers where zero = false, non-zero = true

Definition 1.12. Void: Void is a type without a value and can be used for raw memory pointers or functions without returnvalue.

Definition 1.13. enums: used to enumerate/sequentially assign values to variables.

Code Example 1.14

```
eum { ThisIsZero, ThisIsOne, ThisIsTwo} numbers;
```

1.4 Operators

Code Example 1.15

```
() []
! ~ ++ -- + - * & (type) sizeof
* / %
+ -
<< >>
< <= > >=
== !=
&
^
|
&&
||
?:
= += -= *= /= %= &= ^= |= <<= >>=
,
```

Remark 1.16. Early termination in C means expressions such as A & B never evaluate B if A is already evaluated to false.

Definition 1.17. Ternery Operator: ternery operators allow for concise conditional assignment.

Code Example 1.18

```
int a = {test}? ifTrue : ifFalse;
```

Definition 1.19. Assignment Operator: In C an assignment to a variable is not just a statement its an expressison that evalueates to the value that is assigned.

Definition 1.20. Post- & Pre-Increment/Decement: As in many languages the difference between $a++$ and $++a$ is the value of the expression (before/after incrementing)

Important 1.21. Casting of most types to most other types in possible in C. What exactly happens is very complicated and determined by the types involved.

1.5 Arrays

Definition 1.22. Arrays are finite vectors of variables which all have the same type.

Important 1.23. The C-compiler does not check array bounds, always check them manualy!

Remark 1.24. Arrays do not have to be initialized when they are definied.

Code Example 1.25

```
int x[N] //uninitialized
int x[N] {} //initialized with zeros
```

Remark 1.26. Strings are initialized as an array of chars

Code Example 1.27

```
char s1[6] = "hello";
char s2[6] = {'h', 'e' , 'l', 'l', 'o', 0};
```

Remark 1.28. There are many libraries, to copy, compare, concatenate... strings in C. The language itself does not support any of these operations.

1.6 Preprocessing

Definition 1.29. Preprocessing replaces tokens in a C source file before compilation occurs. Text replacements and macros are managed using directives beginning with #.

Code Example 1.30

```
#define FOO BAZ       // Replaces FOO with BAZ
#define BAR(x) (x+3)  // Replaces BAR(x) with (x+3)
#undef FOO            // Undefines FOO; no replacements after this
    point
#define QUX           // Defines QUX as a null string
```

Remark 1.31. Complex multi-statement macros wrap code in a `do { ... } while(0)` loop to safely absorb trailing semicolons and prevent syntax errors when used in control flow statements (such as `if-else`).

Remark 1.32. Conditional compilation directives include or exclude blocks of code based on conditions evaluated by the preprocessor:

Code Example 1.33

```
#if expression
    ...
#elif expression
    ...
#else
    ...
#endif

#ifdef FOO     // Shorthand for #if defined(FOO)
#ifndef BAR    // Shorthand for #if !defined(BAR)
```

Remark 1.34. Macro parameters can be manipulated using special operators:

- `#c` (Stringifying): Converts a macro parameter into a string literal.
- `x ## y` (Token Pasting / Concatenation): Merges two tokens into a single token at compile-time.

Remark 1.35. The C preprocessor provides several standard predefined macros:

- `__FILE__`: Name of the current source file.
- `__LINE__`: Current line number.
- `__DATE__` / `__TIME__`: Date and time when the preprocessor runs.
- `__STDC__` / `__STDC_VERSION__`: Standard C compliance and version info.

1.7 Modularity

Definition 1.36. Declaration vs. Definition:

- A **declaration** specifies that an entity (function prototype, type, or global variable) exists somewhere without allocating memory for it.
- A **definition** provides the actual implementation or reserves memory for the variable/function.

Definition 1.37. A **module** consists of a public interface defined in a header file (`module.h`) containing declarations and macros, and an implementation in a source file (`module.c`) containing definitions and private declarations.

Remark 1.38. Declarations can be annotated with linkage modifiers:

- `extern`: The definition is located elsewhere (in another object module or compilation unit).
- `static`: Limits the visibility/scope of a function or global variable strictly to the current compilation unit.

Code Example 1.39

```
/* file.h */
#ifndef FILE_H
#define FILE_H

// Declarations, function prototypes, typedefs, and macros

#endif // FILE_H
```

1.8 Assertions

Definition 1.40. An **assertion** (`assert(expression)`) is a macro that evaluates a scalar condition at runtime. If the condition evaluates to false (`0`), it prints diagnostic details (file name, line number, expression) to standard error and terminates execution via `abort()`.

Remark 1.41. Assertions are tools for developers to detect program bugs and verify invariants during development. They should not be used to catch standard runtime or user errors (such as failed file opens). Assertions can be completely disabled in production builds by defining the `NDEBUG` macro (`-DNDEBUG`).

1.9 Style

Remark 1.42. Key guidelines for writing clean C code:

- Maintain clear formatting and consistent indentation to highlight block structure.
- Use explicit parentheses when in doubt to avoid operator precedence ambiguities.
- Use descriptive names for global entities and short names for local variables.
- Eliminate magic numbers using `const`, `enum`, or `#define` constants.
- Avoid function-like macros where regular inline functions suffice.

2 Integers

Lemma 2.1. Integers in C can be represented as **signed** (using two's complement) or **unsigned** (as a standard binary string).

Definition 2.2. Two's Complement: The most significant bit (MSB) has a negative weight (-2^{w-1}), while all other bits have positive weights. This allows representation of both positive and negative integers. To negate a number, invert all bits and add 1 ($\sim x + 1$).

Lemma 2.3. Ranges (w bits):

- Unsigned: 0 to $2^w - 1$
- Signed: -2^{w-1} to $2^{w-1} - 1$

Lemma 2.4. Casting between signed and unsigned does not change the underlying bit representation, only how the bits are interpreted.

Remark 2.5. In operations involving both a signed and an unsigned integer, the signed integer is automatically implicitly cast to unsigned.

2.1 Operations

Remark 2.6. Logic operations (`&`, `|`, `^`, `~`) operate bitwise. They behave as expected and return a signed integer if both operands are signed.

Definition 2.7. Shift Operations:

- **Left Shift** (`x << y`): Shifts bits left by y positions, filling with 0s on the right. Equivalent to multiplying by 2^y.
- **Logical Right Shift** (`x >> y`): Used for unsigned integers. Shifts right by y positions, filling with 0s on the left.
- **Arithmetic Right Shift** (`x >> y`): Used for signed integers. Shifts right by y positions, filling with the sign bit on the left.

Lemma 2.8. Addition: Signed and unsigned addition are performed at the bit level identically. If the result exceeds w bits, it overflows, and the extra bits are truncated.

Lemma 2.9. Multiplication: Exact multiplication requires $2w$ bits. In C, results are truncated to w bits. The bit-level operations for signed and unsigned multiplication yield the same truncated bit pattern.

Remark 2.10. Using Shift for Multiplication/Division:

- Multiplication by a power of 2 ($x \cdot 2^y$) can be optimized as `x << y`.
- Division by a power of 2 ($x/2^y$) can be optimized as `x >> y`. For signed negative integers, a bias is added before shifting to round towards zero: `(x + (1 << y) - 1) >> y`.

Lemma 2.11. Division: Integer division truncates towards zero. It is an expensive operation compared to addition or multiplication.

3 Pointers

3.1 Memory Layout and Virtual Address Space

Definition 3.1. The operating system provides each process with its own private virtual address space. Memory is organized into specific segments:

- **Kernel Virtual Memory:** Memory at the top of the address space that is inaccessible to user code.
- **User Stack:** Dynamically created region at runtime for managing function calls, growing downward.
- **Shared Libraries & Heap:** Memory-mapped region for shared libraries and dynamically allocated memory created via `malloc`.
- **Read/Write Segment:** Stores initialized and uninitialized global and static variables (`.data`, `.bss`) loaded from the executable.
- **Read-Only Segment:** Contains executable code and read-only constants (`.init`, `.text`, `.rodata`) loaded from the executable.

Remark 3.2. Set-up code allocates a stack frame upon entering a procedure to store local variables, return information, and temporary space. The space is deallocated by finish code upon return.

3.2 Pointer Fundamentals

Definition 3.3. A pointer is a variable whose value is the virtual memory address of another variable.

- **Address-of Operator (`&`):** Returns the memory address of a variable.
- **Dereference Operator (`*`):** Accesses or modifies the memory location referred to by a pointer.
- **Double Pointer:** A variable holding the memory address of another pointer (e.g., `int **dp`).
- **NULL Pointer:** A pointer assigned to address `0` (or `0x00000000`), representing a guaranteed invalid memory location.

Important 3.4. Any attempt to dereference a `NULL` pointer results in a segmentation fault.

Code Example 3.5

```
1 int x = 42;
2 int *p = &x;   // p holds the address of x
```

```
int **dp = &p; // dp holds the address of p
*p = 99;       // dereferences p to assign 99 to x
```

3.3 Pointer Operations

Definition 3.6. Pointer Arithmetic: Adding or subtracting an integer n to a pointer shifts the target memory address by $n \times$ sizeof(type) bytes.

Definition 3.7. Casting: Type casting a pointer reinterprets the byte-level data at that memory address according to the new type without changing the address itself.

Code Example 3.8

```
int arr[3] = {1, 2, 3};
int *int_ptr = &arr[0];
int_ptr += 1; // moves forward by sizeof(int) bytes to &arr[1]

char *char_ptr = (char *) int_ptr; // casts int* to char*
char_ptr += 1; // moves forward by sizeof(char) byte
```

3.4 Function Pointers

Definition 3.9. A function pointer stores the executable code address of a function, allowing functions to be passed as arguments or stored inside structures to implement polymorphism and object-oriented patterns.

Code Example 3.10

```
// Pointer to a function taking (int*, char) and returning int
int (*func)(int *, char);

// Example struct containing function pointers (vtable style)
struct file_operations {
ssize_t (*read) (struct file *, char *, size_t, loff_t *);
ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
};
```

4 Dynamic Memory Allocation

Definition 4.1. Static vs Automatic Memory Allocation: Global variables are statically allocated when the program is loaded and deallocated when it exits, while function variables are automatically allocated on function call and deallocated on return.

Remark 4.2. Note that because automatically allocated memory is created on function calls, it is stored on the stack.

Remark 4.3. Often we need a larger amount of memory space that persists across multiple function calls without lasting the entire program lifetime.

Definition 4.4. Dynamic Memory Allocation: We can use allocation functions (such as `malloc`) to explicitly request dynamic memory blocks on the heap.

Code Example 4.5

```
char *a = (char *) malloc(10 * sizeof(char)); // Allocates
    uninitialized memory for 10 characters
char *zeros = (char *) calloc(10, sizeof(char)); // Allocates zero-
    initialized memory for 10 characters
free(a); // Frees allocated memory block

zeros = (char *) realloc(zeros, 20 * sizeof(char)); // Resizes
    allocated memory block to 20 characters
```

4.1 Data Structures

Definition 4.6. Structured Data is a custom type containing multiple data fields of different types. It is defined using the `struct` keyword.

Code Example 4.7

```
struct myData {
    int dataField;
    char dataName;
};
```

Definition 4.8. Unions overlay multiple member fields in a single shared memory location. Only one member value should be stored/interpreted at a time, and C

performs no runtime tracking of the active member type.

Code Example 4.9

```
union currentTime {
    char stringName[10];
    int intTime;
    long longTime;
};
```

Remark 4.10. We can use `typedef` to define type aliases. Mixing struct tags and typedef aliases with the same name can cause confusion.

Remark 4.11. Tags (e.g., `struct tag` {...}) belong to a separate namespace from typenames; consequently, a typename and a tag can share the same identifier.

Remark 4.12. Structs and pointers allow building **dynamic data structures**. Using `void` pointers allows elements to be generic.

Code Example 4.13

```
struct listNode {
    void *value;
    struct listNode *next;
};

struct listNode *myList = NULL;
```